Adaptive Edge AI Architecture

Research-grounded design and process-flow diagrams | React + Node.js + Python | on-premises-first

Executive conclusion: run low-cost, continuous occupancy and tracking; selectively invoke pose and expensive RGB task recognition; fuse evidence with shift and machine context; publish auditable, reversible candidate events. Do not equate video-only labels with employee intent, payroll time, or verified productivity.

1. Recommended end-to-end architecture

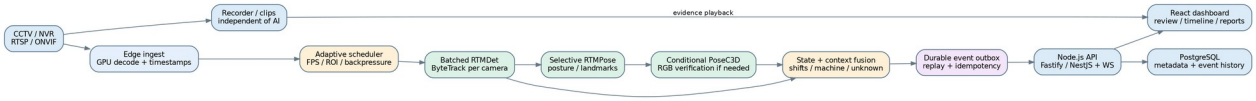


Figure 1. Independent recording and AI inference; conditional deep analysis; local Node.js application.

Deploy one edge appliance per factory or camera group. Keep camera decode and recording independent from AI worker restarts. Python owns computer vision and model inference; Node.js owns identity authorization, configuration, review workflows, integrations and reporting. Prefer a modular monolith for the Node.js backend and a small durable outbox over a premature Kafka cluster. The design is an engineering synthesis, not a benchmarked end-to-end result.

2. Dynamic inference policy

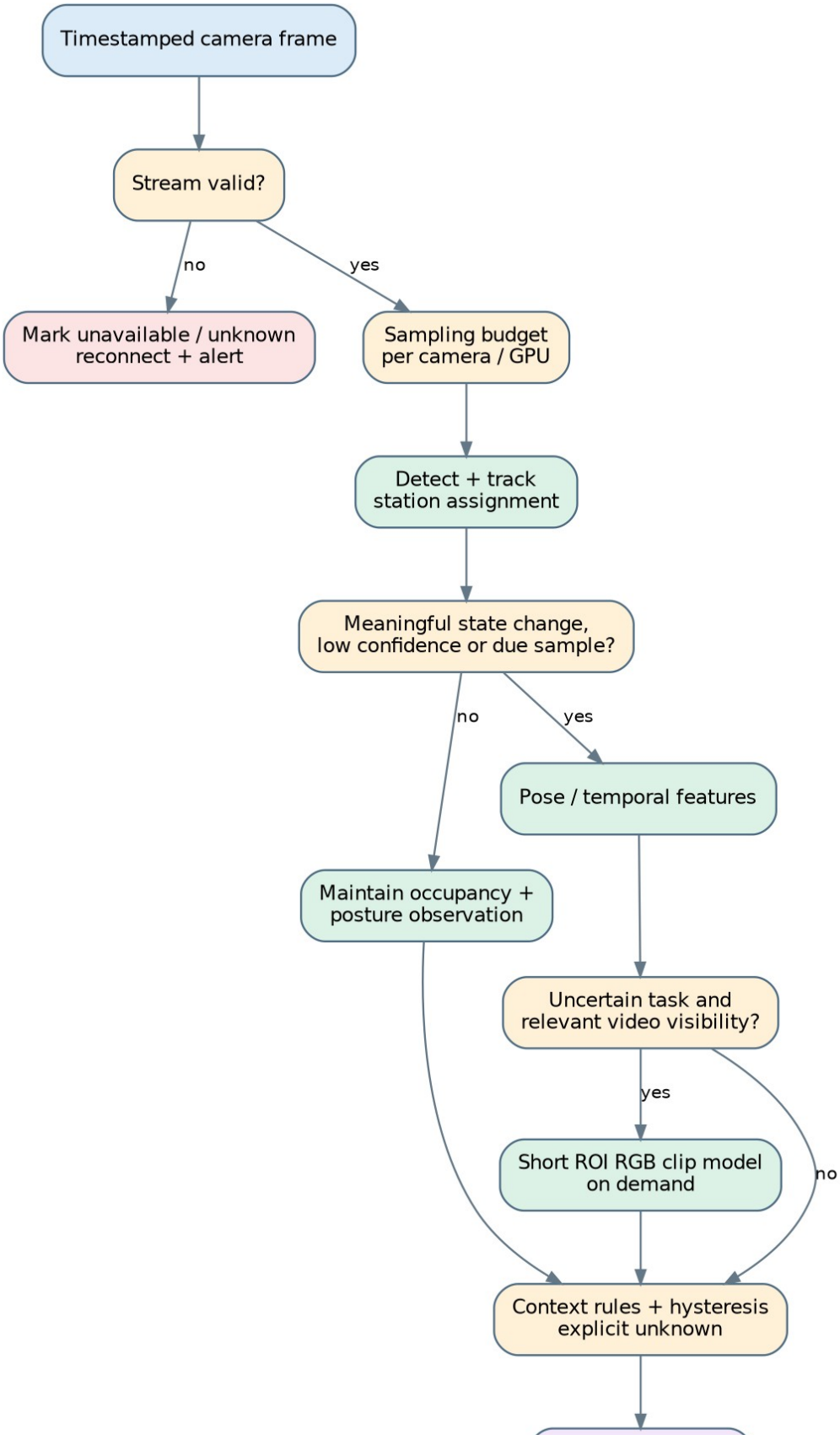

Timestamped camera frame
Stream valid?
no
yes
Mark unavailable / unknown
reconnect + alert
Sampling budget
per camera / GPU
Detect + track
station assignment
Meaningful state change,
low confidence or due sample?
no
yes
Pose / temporal features
Maintain occupancy +
posture observation
Uncertain task and
relevant video visibility?
yes
Short ROI RGB clip model
on demand
no
Context rules + hysteresis
explicit unknown

Figure 2. Proposed adaptive scheduling flow. Conditions represent engineering policies to validate, not published thresholds.

Sampling baseline: trial 5/10/15 FPS per camera; increase briefly around state transitions or uncertain task evidence; decrease where occupancy is stable and visual change is small. Apply hard fairness limits per camera, GPU utilization caps, queue deadlines and a maximum event detection delay. Never drop the recording stream to save inference cycles. A stale frame or blocked view maps to UNKNOWN, not absence or inactivity.

Pose estimation operates on selected person crops; task models operate on short, relevant station clips. Use ROI caching and decode once when practical. Under GPU pressure, reduce sampling and defer high-cost classifiers while retaining monitoring, coverage accounting and event replay. All policy changes are versioned.

3. Process state flow and human review

Stage	Output / guardrail
Presence	Person and station occupancy with stream-health and visibility state.
Posture	Sitting/standing/walking/unknown with confidence and occlusion handling.
Task	Specific visible action: sewing, handling, packing, inspection, or unknown.
Context	Machine running/stopped, material availability, shifts, permitted breaks.
Candidate	Configurable duration + hysteresis; evidence clip + reason request.
Review	Supervisor confirms process delay, permitted activity, false detection or unresolved.
Reporting	Station-level recoverable interruptions, unknown/coverage denominators, corrections.

Proposed trial rule: when in a production interval, station occupied, visibility adequate and no operation-defined task observed for 60 seconds, generate a reviewable candidate. The 60-second figure is a starting hypothesis only. Camera track IDs are NOT employee IDs; use verified roster/badge assignment for named timelines.

4. Runtime topology and failure recovery

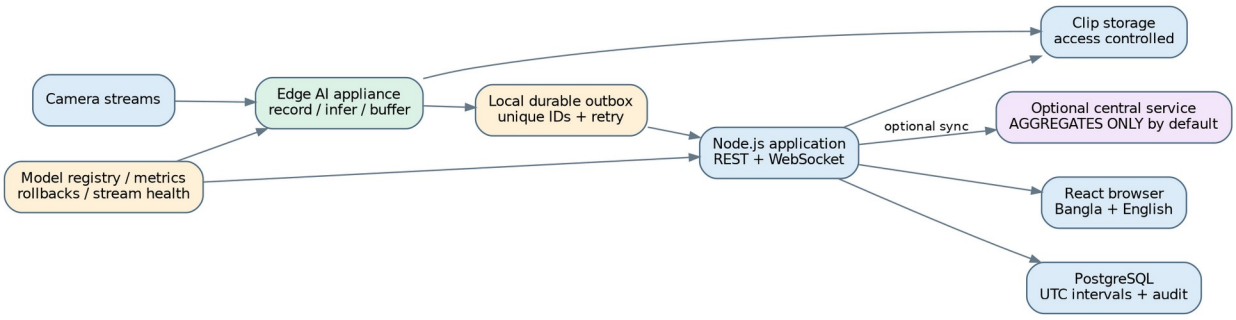


Figure 3. Local-first operation, idempotent event delivery and optional aggregate-only central sync.

Failure and recovery contracts

Fault	Required behavior
Camera disconnected	Heartbeat timeout → stream unavailable; reconnect with bounded backoff; no false idle events.
Model worker crashed	Recording stays up; observations become unknown; supervisor restarts Python worker.
API / network outage	Local append-only outbox persists timestamped observations; deterministic replay without duplicate event IDs.
GPU saturation	Scheduler degrades optional RGB then pose rate before occupancy; report reduced coverage.
Camera moved / model changed	Invalidate zone calibration; log version; canary/rollback and revalidation.
Power outage	UPS with orderly shutdown and restart; prove recovery during pilot.

5. Research comparison and what to combine

Method / paper	Use it for	Reason and caution
RTMDet [R1]	Person detector baseline	Efficient multi-scale real-time detection; COCO benchmarks do not predict factory FPS.
ByteTrack [R2]	Within-camera continuity	Associates lower-confidence detections to recover occluded tracks; not identity verification.
RTMPose [R3]	Selective body landmarks	Optimized pose extraction; hips/knees may be blocked at sewing stations.

ST-GCN [R4]	Low-cost pose-sequence baseline	Good comparative baseline; hands, products and tools may be missed.
PoseC3D [R5]	Preferred pose-model experiment	Heatmap representation shown robust to pose noise in its benchmarks; verify edge cost.
SlowFast [R6]	RGB model reference	Fast motion plus slower appearance; benchmark a compact RGB alternative before adopting.
MS-TCN [R7]	Interval segmentation baseline	Temporal refinement suppresses fragmented predictions; must use causal window online.
ASFormer [R8]	Offline upper-bound experiment	Long-range modeling; avoid treating full-sequence metrics as live latency.
VideoMAE [R9]	Later domain adaptation	Self-supervised pretraining candidate after enough permitted unlabeled footage.
Assembly101 / IKEA ASM / HA4M [R10–R12]	Labeling and experiment design	Domain differs from factory CCTV; check noncommercial dataset and checkpoint terms.

Recommended hybrid model strategy

Baseline A: RTMDet → ByteTrack → calibrated station polygons + time rules. Baseline B: add RTMPose → posture classifier and simple temporal features. Baseline C: add PoseC3D only for task labels that cannot be captured by simpler features. Baseline D: add a compact ROI RGB classifier only for actions distinguished by objects/hands and ambiguous pose outputs. Baseline E: optional causal temporal segmentation after comparing event boundary error to hysteresis rules. Keep the lowest-cost configuration that meets the jointly agreed error and coverage gates.

6. Application architecture: no .NET

Layer	Pilot choice	Boundary
Web client	React + TypeScript; TanStack Query; WebSockets	Dashboard, role-scoped views, timeline, clip review and localization.
Backend	Node.js + TypeScript; Fastify or NestJS	REST, authentication, RBAC, audit, settings, event review, exports, WS fan-out.
Inference	Python + PyTorch / ONNX Runtime / TensorRT where suitable	RTMDet, ByteTrack, RTMPose and optional action workers.
Media	GStreamer / FFmpeg;	RTSP ingest, stream health,

	benchmark DeepStream on NVIDIA	frame sampling, recordings, encoded clip evidence.
Database	PostgreSQL	Stations, cameras, observations, review outcomes, rule/model versions; UTC timestamps.
Durable delivery	Postgres outbox or local disk-backed queue	At-least-once delivery + idempotency; do not queue raw frames.
Deployment	Docker Compose initially	Per-site rollout and health; move to orchestration only at measured need.

Data contract: factory_id, camera_id, station_id, track_id, optional verified_employee_id, observed_at_utc, interval_start/end, visibility, posture, task, confidence, model_version, rule_version, evidence_uri, revision, reviewer and review_status. Raw video stays in controlled media storage, not Postgres. WebSocket clients receive summarized state deltas, not full-rate pose outputs.

7. Performance and accuracy validation

Metric	Pilot gate / policy
Continuous monitoring	At least 95% usable occupancy coverage in approved visible zones — proposed goal.
Posture	Macro-F1 ≥ 0.90 on visible sitting/standing cases — proposed goal.
Candidates	Precision ≥ 90%, recall ≥ 80%, max 1 false candidate/station/8-hour shift — proposed goals.
Operational	Measure sustained camera count, p95 end-to-end latency, GPU utilization, drop rate, reconnect and recovery.
Temporal	Measure segment-F1, boundary MAE and duration MAE separately from frame classification.
Dynamic versus fixed	A/B the adaptive scheduler against fixed 5, 10 and 15 FPS at equal footage and acceptable error; report compute-hours and false alerts.
Generalization	Held-out days, sessions, camera views and if possible a second site; no adjacent-frame leakage.

Do not claim a specific percentage reduction in compute without benchmarking. Start with 2–4 streams and 5–10 stations; measure person crops/s, clip invocation rate, p95 GPU queues and error trade-offs before sizing a multi-camera appliance.

8. Twelve-week implementation sequence

Weeks	Deliverable
1–2	Factory/video feasibility, station zones, ontology, consent and retention policies; baseline test set.
3–4	RTSP ingest, recording separation, RTMDet + ByteTrack, stream-health, basic React occupancy UI.
5–6	RTMPose posture, uncertain states, label agreement and segmented held-out evaluation.
7–8	Adaptive scheduler, compare PoseC3D vs features and selective RGB; add shift/machine context.
9–10	Durable event/replay, review workflow, shadow mode, stress and outage testing.
11–12	Locked performance evaluation, calibration, compute cost per camera and release decision.

9. Source bibliography and verification status

This technical architecture draws on the supplied 28 October 2026 Factory Vision AI plan and checks primary/public abstracts or publication records below. I reviewed the published methods and design claims available on their source pages; this is not a reproducibility audit or a claim to have reproduced all reported experiments.

R1 RTMDet — https://arxiv.org/abs/2212.07784

R2 ByteTrack — https://www.ecva.net/papers/eccv_2022/papers_ECCV/html/315_ECCV_2022_paper.php

R3 RTMPose — https://arxiv.org/abs/2303.07399

R5 PoseC3D — https://openaccess.thecvf.com/content/CVPR2022/html/Duan_Revisiting_Skeleton-Based_Action_Recognition_CVPR_2022_paper.html

R6 SlowFast — https://openaccess.thecvf.com/content_ICCV_2019/html/Feichtenhofer_SlowFast_Networks_for_Video_Recognition_ICCV_2019_paper.html

R7 MS-TCN — https://openaccess.thecvf.com/content_CVPR_2019/html/Abu_Farha_MS-TCN_Multi-Stage_Temporal_Convolutional_Network_for_Action_Segmentation_CVPR_2019_paper.html

S1 NVIDIA DeepStream adaptive batching — https://docs.nvidia.com/deepstream/text/DS_plugin_gst-nvstreammux2

Additional methods R4 ST-GCN, R8 ASFormer, R9 VideoMAE and dataset resources R10–R12 are described in the original supplied research plan; include their full-paper and licensing reviews in the experimental sprint before implementation choices are locked. Commercial data/weight permissions must be verified separately.